Lecture 12
Single cycle processor design

Name	Roll Number
Sujal Meshram	CS23BTECH11060
Morankar Kaushal Nandkumar	CS23BTECH11037
Arnav AB Maiti	CS23BTECH11007
Karan Gupta	CS23BTECH11023

2 September, 2024

Contents

1 Introduction **3**

2 Datapath and Control **3**
2.1 Datapath . . . 3
2.2 Control . . . 3

3 Datapath for R Format **3**
3.1 Example of R-Format Datapath Operation . . . 3
3.2 Datapath Diagram . . . 4

4 Load/Store Instructions **4**
4.1 Datapath Diagram . . . 5
4.2 Operation Example . . . 5

5 Branch Instructions **6**
5.1 Datapath Diagram . . . 6
5.2 Operation Example . . . 6

6 Full Datapath without control **7**

7 ALU Control **8**
7.1 ALU Control Signal Derivation . . . 8

8 The Main Control Unit **9**
8.1 Control Signals and Their Interpretation . . . 9
8.2 Instruction Decoding . . . 9
8.2.1 Decoding Examples . . . 9
8.2.2 ALU Control Logic . . . 10

9 Full Datapath With Control **10**

10 Performance Issues **11**
10.1 Improving Performance: Pipelining . . . 11

1 Introduction

This part discusses building a hardware design for a RISC-V processor by connecting different digital blocks together. Refer to the diagrams more to have an intuition about the work being done by the processor.

2 Datapath and Control

Any digital logic design consists of a datapath and a control, as explained below.

2.1 Datapath

The datapath consists of connections between various components of a CPU which indicates possible paths for the data to flow from one point to another. It comprises of various components that work together to execute instructions. The datapath can vary depending on the ISA and the format of instructions. Key components of the datapath includes Register File, Registers, Arithmetic Logic Units (ALUs), Multiplexors (MUXes), Memory and Gates.

2.2 Control

The control unit controls the operations of the datapath or the path through which the data actually flows based on the instructions. Control signals are often derived from the opcode of the instruction. Important control signals include:

- **Select lines for Multiplexors:** Determine which input is selected for the multiplexor to forward.
- **ALU Control Signals:** Specify the type of operation that the ALU should perform.
- **Register File/Memory Write Signals:** Indicate whether register should be written to or not.

The subsequent sections will discuss the datapath and control design for a selected subset of instructions:

1. arithmetic/logical instructions (add,sub,and,or)
2. memory reference instructions (ld, sd)
3. control transfer (beq)

---

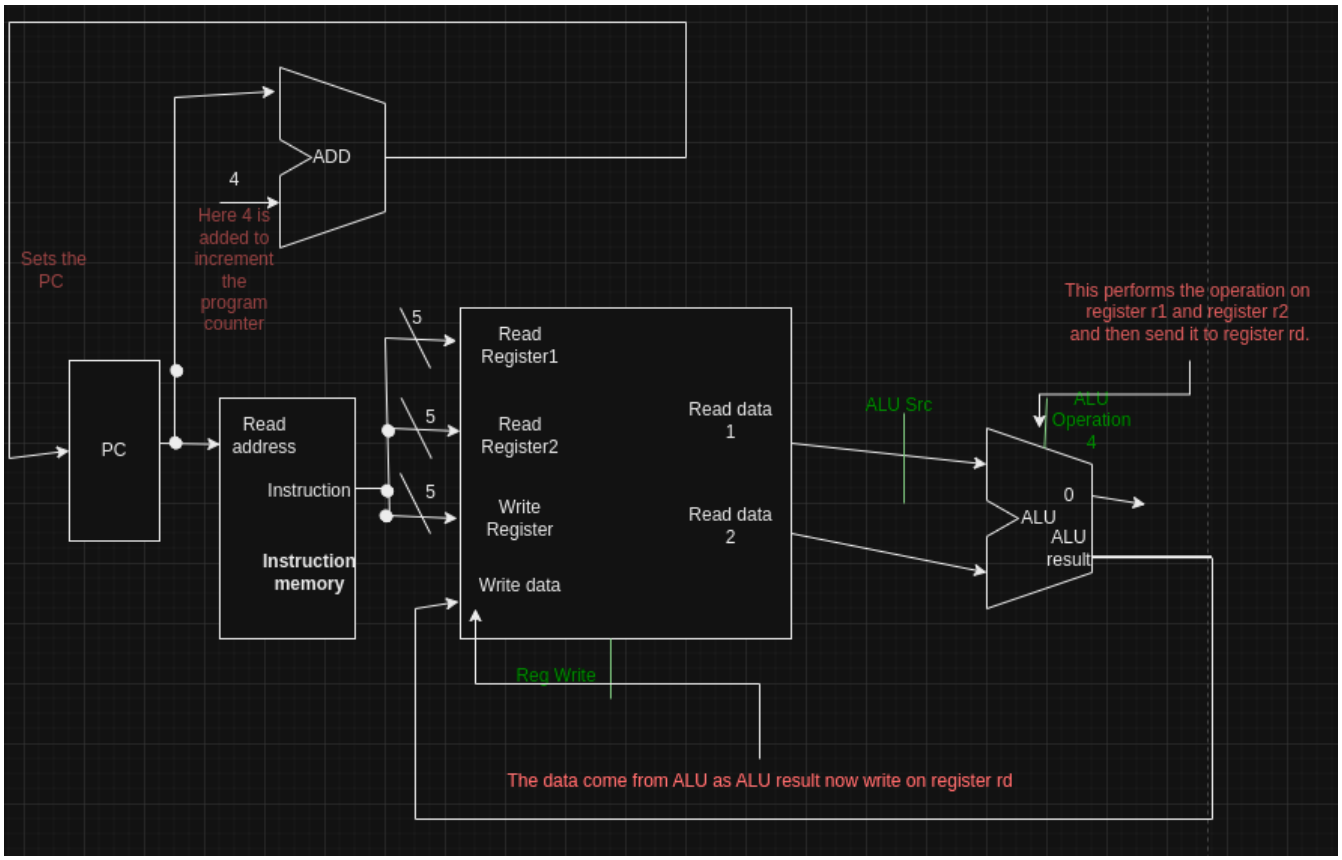
3 Datapath for R Format

R-format instructions are used for operations that involve only registers. The datapath for these instructions is designed to handle arithmetic and logical operations where both operands and the result are registers.

3.1 Example of R-Format Datapath Operation

For the R-format instruction `add rd, rs1, rs2`:

3.2 Datapath Diagram



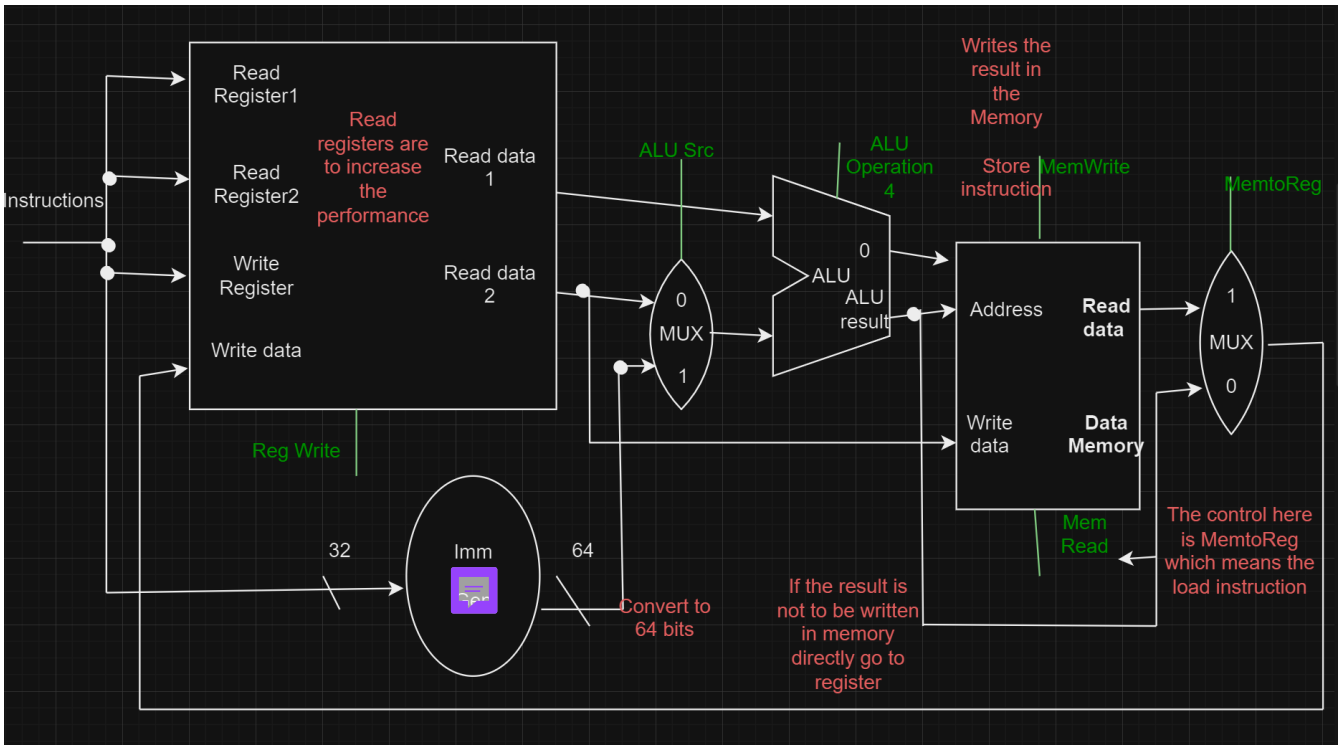
1. **Instruction Fetch:** The `add` instruction is fetched from memory.
2. **Register Read:** The values of registers `rs1` and `rs2` are read from the Register File.
3. **ALU Operation:**
   - The values from `rs1` and `rs2` are sent to the ALU.
   - The ALU Control Unit configures the ALU to perform addition based on the control value.
4. **Result Write Back:** The result of the ALU operation is written to the destination register `rd`.

---

4 Load/Store Instructions

Load/store instructions in RISC architectures are used for accessing memory. These instructions operate by either loading data from memory into a register or storing data from a register into memory.

4.1 Datapath Diagram



4.2 Operation Example

For a load instruction `ld rs2, offset(rs1)`:

1. **Instruction Fetch:** The `ld` instruction is fetched and decoded.
2. **Register Read:** The base address is read from register `rs1`.
3. **Address Calculation:**
   - The ALU computes the effective address by adding the base address and the offset.
4. **Memory Read:**
   - The effective address is used to read data from memory.
5. **Write Back:**
   - The data read from memory is written to the destination register `rs2`.

For a store instruction `sd rs2, offset(rs1)`:

1. **Instruction Fetch:** The `sd` instruction is fetched and decoded.
2. **Register Read:** The base address is read from register `rs1`, and the data to be stored is read from register `rs2`.
3. **Address Calculation:**

- The ALU computes the effective address by adding the base address and the offset.

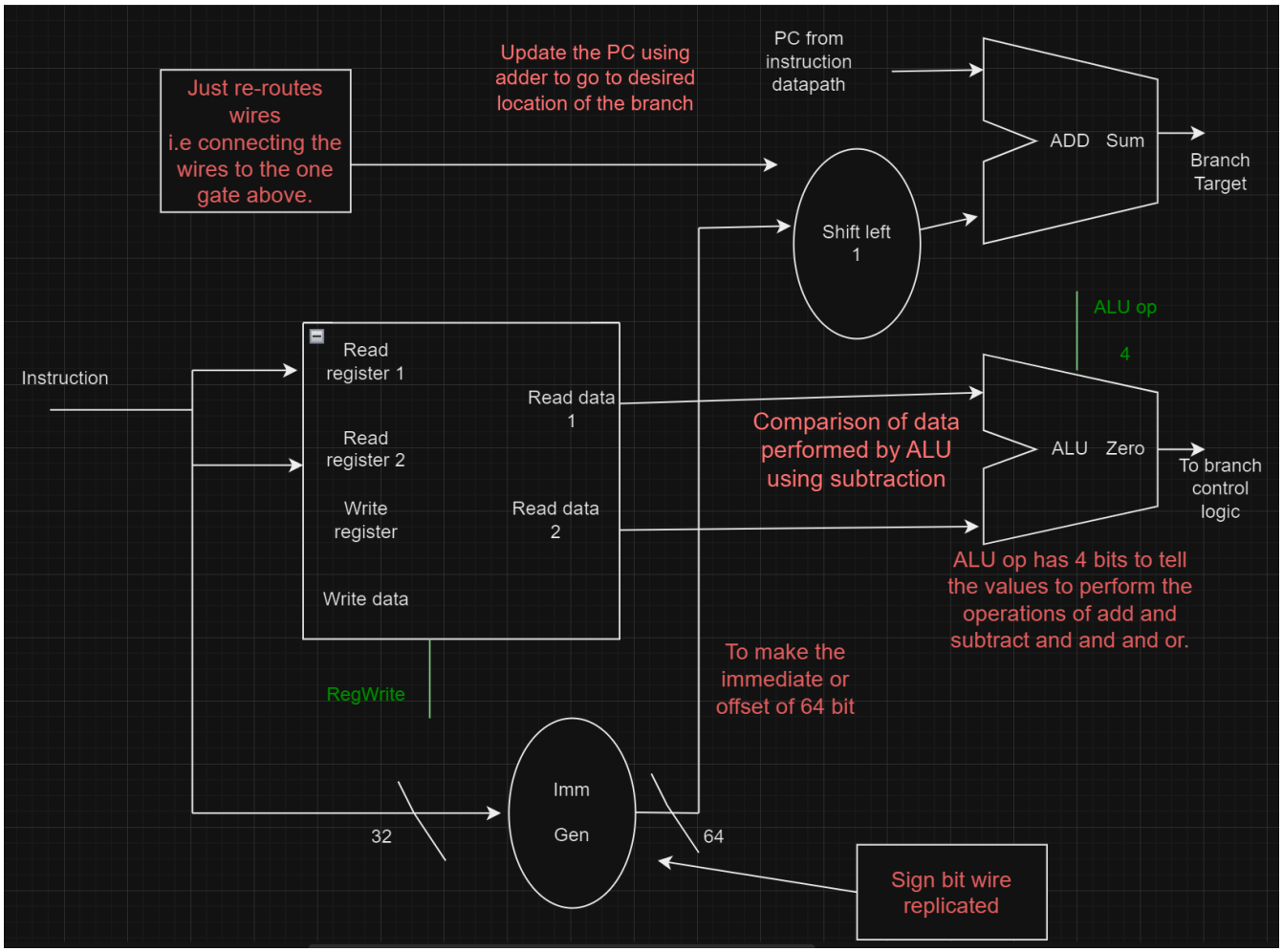
4. **Memory Write:**
   - The data from register `rs2` is written to memory at the computed effective address.

---

5 Branch Instructions

The branch instructions are used to compare the values of two registers and branch to a target address if they satisfy a certain condition. The datapath for these instructions is designed to update the program counter based on the target address.

5.1 Datapath Diagram



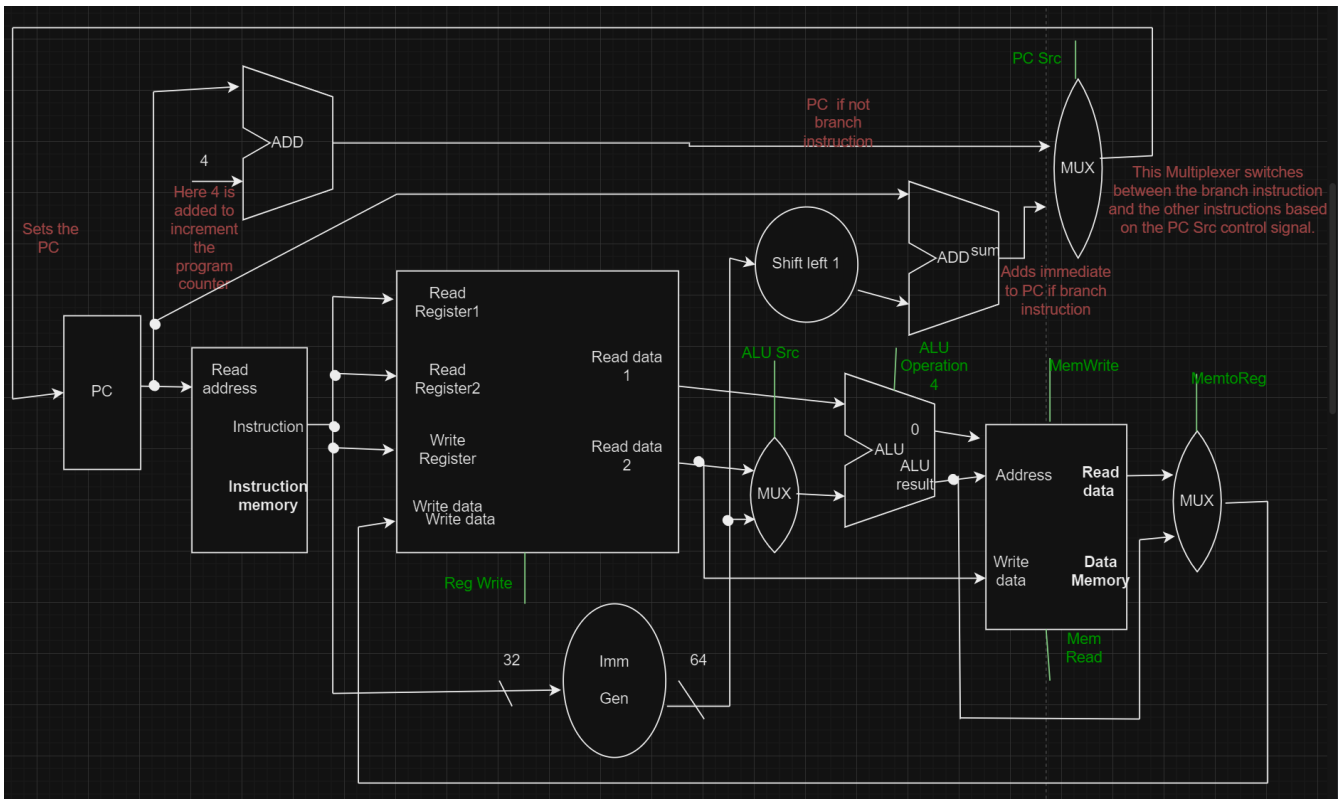
5.2 Operation Example

For a branch instruction `beq rs1, rs2, offset`:

1. **Instruction Fetch:** Fetch the `beq` instruction from memory.
2. **Register Read:** Read the values from registers `rs1` and `rs2`.
3. **Comparison:**
   - The ALU compares the values of `rs` and `rt`.
   - The ALU produces a signal indicating whether the values are equal.
4. **Branch Decision:**
   - The Branch Control Unit checks the equality signal from the ALU.
   - If the values are equal, compute the immediate, which is given in the instruction and shift it left by 1 bit as the instruction is B-Format.
5. **PC Update:**
   - Update the Program Counter to the computed branch target address if the branch is taken.
   - Otherwise, continue with the next instruction.

---

6 Full Datapath without control



The above model is to just show the overview of the single cycle processor how the functions work according to the paths they are joined.

---

7 ALU Control

In RISC-V architecture, the ALU control signals are derived from the opcode of the instruction and the ALUOp signal. The following sections describe how the ALU control signals are determined based on these inputs and the type of instruction.

7.1 ALU Control Signal Derivation

Assuming a 2-bit **ALUOp** is derived from the opcode of the instruction, the combinational logic generates the ALU control signals.

opcode	ALUOp	Operation	Opcode/func7, func3 fields	ALU Function	ALU Control
ld	00	load register	XXXXXXXXXX	add	0010
sd	00	store register	XXXXXXXXXX	add	0010
beq	01	branch on equal	XXXXXXXXXX	subtract	0110
R-Type	10	add	0x00, 0x0	add	0010
		subtract	0x20, 0x0	subtract	0110
		and	0x00, 0x7	and	0000
		or	0x00, 0x6	or	0001

The ALU is used for different types of operations, as follows:

Operation Types and Control Signals

- **Load/Store Instructions**:
  - **ALUOp**: 00
  - **Operation**: Addition
  - **Control Signal**: ALU control signal is set to the code corresponding to addition (e.g., 0010).
- **Branch Instructions**:
  - **ALUOp**: 01
  - **Operation**: Subtraction
  - **Control Signal**: ALU control signal is set to the code corresponding to subtraction (e.g., 0110).
- **R-Type Instructions**:
  - **ALUOp**: 10
  - **Operation**: Depends on the **funct** bits (i.e., **funct3** and **funct7**)
    - **0000**: Logical **AND**
    - **0001**: Logical **OR**
    - **0010**: **ADD**
    - **0110**: **SUB**

8 The Main Control Unit

In the RISC-V architecture, the **ALUOp** signals, along with the **funct3** and **funct7** fields, are used to determine the specific ALU operation to perform.

ALUOp		**Funct7 field**							**Funct3 field**			**Operation**
ALUOp1	**ALUOp0**	**I[31]**	**I[30]**	**I[29]**	**I[28]**	**I[27]**	**I[26]**	**I[25]**	**I[14]**	**I[13]**	**I[12]**	
0	0	X	X	X	X	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	X	X	X	X	0110
1	X	0	0	0	0	0	0	0	0	0	0	0010
1	X	0	1	0	0	0	0	0	0	0	0	0110
1	X	0	0	0	0	0	0	0	1	1	1	0000
1	X	0	0	0	0	0	0	0	1	1	0	0001

8.1 Control Signals and Their Interpretation

- **ALUOp1 = 1**: This signal indicates that the ALU operation is determined by the values of the **funct3** and **funct7** fields. These fields help specify whether the operation is **ADD**, **SUB**, **OR**, etc.
- **ALUOp0 = X**: This signal is a don't care condition in this scenario because the instruction type (like R-type) is fully determined by the combination of the opcode, **ALUOp1**, and the fields **funct3** and **funct7**.

8.2 Instruction Decoding

When **ALUOp1 = 1**, the decoder logic examines the **funct3** and **funct7** fields to distinguish between different ALU operations. The control unit uses these fields as follows:

Instruction Fields

- **Opcode (0110011)**: Identifies the instruction as an R-type instruction.
- **funct3**: These 3 bits narrow down the operation within the ALU category (e.g., **ADD**, **SUB**, **OR**).
- **funct7**: These 7 bits further distinguish between operations such as **ADD** and **SUB**, or between arithmetic and logical shifts.

8.2.1 Decoding Examples

Instruction	**Opcode**	**funct3**	**funct7**	**4 Bits of ALU**	**Operation**
ADD	0110011	000	0000000	0010	Register addition
SUB	0110011	000	0100000	0110	Register subtraction
OR	0110011	110	0000000	0001	Bitwise OR
AND	0110011	111	0000000	0000	Bitwise AND

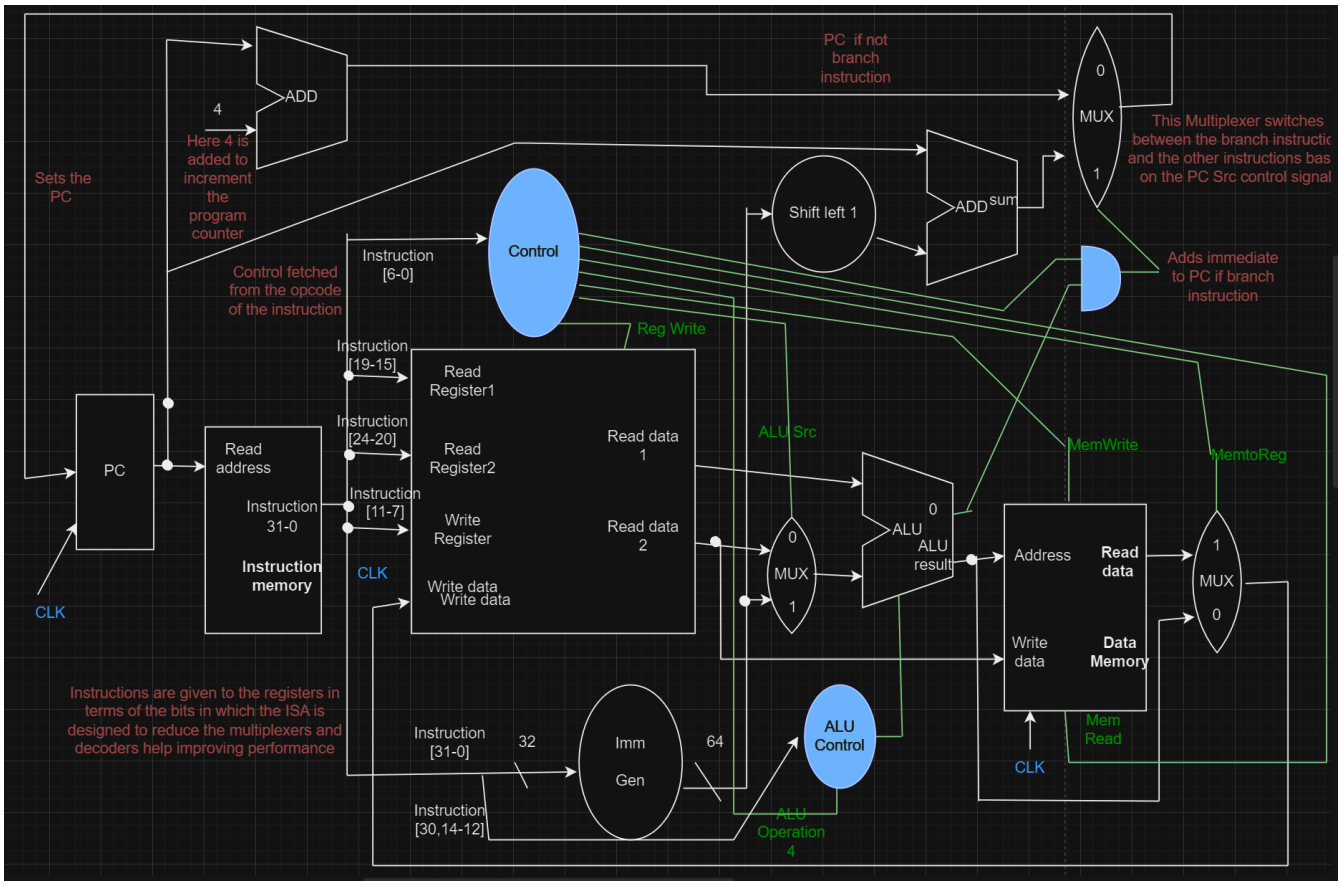
Table 1: Decoding ALU Operations

8.2.2 ALU Control Logic

The **ALUOp1** signal directs the ALU control unit to consider the **funct3** and **funct7** fields. The control unit uses combinational logic or a lookup table to map specific combinations of **funct3** and **funct7** to different ALU operations. Based on these fields, the ALU performs operations like **ADD**, **SUB**, **OR**, or others.

---

9 Full Datapath With Control



The above model demonstrates the processor in a single cycle where each instruction is processed according to the control signals based on the opcode (instructions[6-0]).

Example:

1. **R-format:** Doesn't require immediate generation or the branching part, which is controlled by signals.
2. **Load instruction:** Requires the immediate generation part where the 32-bit value is extended to a 64-bit number by extending the MSB. Here, the load instruction uses `MemRead` to read from memory, and the immediate value is written to the register using `MemToReg`.
3. **S-format:** Takes both the immediate and memory access into account to store the data in memory, using the signal `MemWrite`.

4. **B-format:** Uses the branch part, i.e., the ADD and shift left by 1 operation, where the control ensures that the PC is incremented as per the offset.

---

10 Performance Issues

In a single-cycle RISC-V processor, the longest delay determines the clock period. The critical path for the load instruction is as follows:

- Instruction Memory → Register File → ALU → Data Memory → Register File

This long critical path means that the clock period is fixed by the slowest instruction, which leads to inefficiencies. It is not feasible to vary the clock period for different instructions, as this would complicate the design and introduce new challenges in maintaining timing.

One of the key design principles violated by this approach is:

- **Making the common case fast**: In a single-cycle processor, all instructions take the same amount of time, regardless of their complexity. For example, even simple instructions like register-register add operations take as long as load/store instructions, which have a much longer execution path.

10.1 Improving Performance: Pipelining

To overcome this limitation, we can improve performance by introducing pipelining. In a pipelined processor, multiple instructions are overlapped in execution, and each stage of instruction execution (e.g., instruction fetch, decode, execute, memory access, write-back) takes place in separate pipeline stages.

By pipelining, the following benefits are achieved:

- Shorter clock period: Each pipeline stage only needs to complete part of the instruction execution, reducing the delay of the critical path.
- Higher instruction throughput: Multiple instructions are executed in parallel across different pipeline stages.

Thus, pipelining addresses the inefficiencies of a single-cycle processor by making the common case faster and allowing more efficient execution of instructions.
